Question Bank — Week 2

Complexity and the Array · control flow, functions, errors (Lecture 02, Lab 02) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **2**

Part B — Short answer and essay **9**

Part C — Complexity analysis **10**

Part D — Proofs **11**

Part E — Array state **11**

Part F — Trace the code **12**

Part G — Find and fix the bug **13**

Part H — Write the code **14**

> ***Answers are in a separate file:*** `week02-answers.md`. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. Unless a question says otherwise, n is the input size and* `Array` *is the course array from* `dsa/array.py`.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W2-M01 – W2-M44
B	Short answer and essay	W2-E1 – W2-E10
C	Complexity analysis — give Θ and justify	W2-K1 – W2-K6
D	Proofs	W2-P1 – W2-P3
E	Array state — show every step	W2-S1 – W2-S4
F	Trace the code	W2-T1 – W2-T4
G	Find and fix the bug	W2-B1 – W2-B3
H	Write the code — checked by `pytest`	W2-C1 – W2-C4

Part A — Multiple choice

W2-M01 [how] $f(n) = 3n^2 + 5n + 2$ is:

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(n^3)$
- **d)** $\Theta(n \log n)$

W2-M02 [why] Which statement is **true**?

- **a)** $n^2 = O(n)$
- **b)** $n = \Omega(n^2)$
- **c)** $n = O(n^2)$
- **d)** $2^n = O(n^3)$

W2-M03 [what] $f(n) = O(g(n))$ means:

- **a)** there are constants $c > 0$ and n_0 with $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- **b)** $f(n) \leq g(n)$ for every n
- **c)** $f(n) = g(n)$ for all large n
- **d)** there are constants $c > 0$ and n_0 with $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

W2-M04 [how] In the **worst case**, how many comparisons does linear search make on an array of n elements?

- **a)** 1
- **b)** n / 2
- **c)** $\log_2 n$
- **d)** n

W2-M05 [how] The target is present and equally likely to be at any of the n positions. On **average**, how many comparisons does linear search make?

- **a)** $(n + 1)/2$
- **b)** n
- **c)** $\log_2 n$
- **d)** $n^2/2$

W2-M06 [how] What is the running time of this loop?

```
i = n
while i > 1:
    i = i // 2
```

- **a)** $\Theta(1)$
- **b)** $\Theta(n)$
- **c)** $\Theta(\log n)$
- **d)** $\Theta(n/2)$

W2-M07 [how] What is the running time?

```
for i in range(n):
    for j in range(i):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n \log n)$
- **c)** $\Theta(i \cdot n)$
- **d)** $\Theta(n^2)$

W2-M08 [how] What is the running time?

```
for i in range(n):
    for j in range(5):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(5^n)$
- **d)** $\Theta(\log n)$

W2-M09 [what] For a Python `list` of n items, `x in values` costs:

- **a)** $O(1)$, because lists are hash tables
- **b)** $O(n)$
- **c)** $O(\log n)$
- **d)** $O(n^2)$

W2-M10 [what] Which Python `list` operation is $O(1)$ (amortised)?

- **a)** `values.insert(0, x)`
- **b)** `values.pop(0)`
- **c)** `values.append(x)`
- **d)** `values.remove(x)`

W2-M11 [why] Why is reading `a[i]` from an array $O(1)$?

- **a)** The array is sorted
- **b)** Python caches the last element read
- **c)** The array searches from both ends at once
- **d)** The slots are contiguous and equal-sized, so the address is base + i × slot size

W2-M12 [how] A partly filled array has **size 6**. How many elements must move to insert a new value at **index 2**?

- **a)** 4
- **b)** 2
- **c)** 6
- **d)** 3

W2-M13 [how] When inserting into an array, the elements after the gap must be shifted right. In which order?

- **a)** From the insertion index towards the end
- **b)** From the last used slot back towards the insertion index
- **c)** Any order — the result is the same
- **d)** Swap each element with its neighbour, from the front

W2-M14 [how] `a = Array(4)`, then `a[-1]`. What happens?

- **a)** returns the last slot, `None`
- **b)** returns `a[3]`
- **c)** raises `IndexError`
- **d)** raises `TypeError`

W2-M15 [how] `a = Array(5)`, then `a[0] = 7`. What is `len(a)`?

- **a)** 1
- **b)** 0
- **c)** 6
- **d)** 5

W2-M16 [how] A 3 × 4 matrix (3 rows, 4 columns) is stored flat in row-major order. At which index is element (2, 1)?

- **a)** 7
- **b)** 9
- **c)** 6
- **d)** 11

W2-M17 [how] What extra space does `reverse_in_place(arr)` need (swap from both ends, no second array)?

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W2-M18 [why] Under the course storage rule (Lecture 02), which of these is **not** allowed?

- **a)** a `Stack` storing its items in the student's own `DynamicArray`
- **b)** a `Graph` storing adjacency in the student's own `ChainingHashMap`
- **c)** a queue class storing its items in a Python `list`
- **d)** `bfs` returning the visit order as a Python `list`

W2-M19 [what] Python's built-in `list` is best described as:

- **a)** a linked list of values
- **b)** a fixed-size C array of integers
- **c)** a hash table keyed by position
- **d)** a dynamic array of references to objects

W2-M20 [how] An $O(n)$ loop is followed by an $O(n^2)$ loop. The whole is:

- **a)** $O(n^2)$
- **b)** $O(n^3)$

- **c)** $O(n)$
- **d)** $O(n+n)$

W2-M21 [how] `values` is a Python list of n items. The cost of:

```
for i in range(len(values)):
    if values[i] in values[i + 1:]:
        return True
```

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(1)$

W2-M22 [what] When does the `else` block of a `for` loop run?

- **a)** Whenever the loop body raises an exception
- **b)** Only if the loop runs zero times
- **c)** When the loop finishes without executing `break`
- **d)** After every iteration

W2-M23 [how] Given

```
def f(a, L=[]):
    L.append(a)
    return L
```

what does `f(1)` then `print(f(2))` print?

- **a)** `[2]`
- **b)** `[1]`
- **c)** `[2, 1]`
- **d)** `[1, 2]`

W2-M24 [how] What is the value of `0 or 'x' and ''`?

- **a)** `''`
- **b)** `'x'`
- **c)** `0`
- **d)** `False`

W2-M25 [how] `list(range(2, 11, 3))` is:

- **a)** `[2, 5, 8, 11]`
- **b)** `[2, 5, 8]`
- **c)** `[3, 6, 9]`
- **d)** `[2, 3, 4, 5, 6, 7, 8, 9, 10]`

W2-M26 [how] What does `print(g())` print?

```
def g():
    try:
        return 1
```

```
    finally:
        print("bye")
```

- **a)** `1`
- **b)** bye
- **c)** bye then `1`
- **d)** `1` then bye

W2-M27 [what] In the RAM model of Lecture 02, which of these is **not** one basic step?

- **a)** `a[i] = v` on an array
- **b)** `x in values` on a Python list of n items
- **c)** `x < y` on two small integers
- **d)** calling a function (the call itself, not what it then does)

W2-M28 [how] An algorithm does n^2 basic steps and takes 1 second at n = 10,000. About how long will it take at n = 100,000, on the same machine?

- **a)** 10 seconds
- **b)** 1,000 seconds
- **c)** 2 seconds
- **d)** 100 seconds

W2-M29 [why] Algorithm A does $1000n$ steps; algorithm B does n^2 steps. Which statement is **true**?

- **a)** B does fewer steps for n < 1,000; A does fewer for every n > 1,000
- **b)** A does fewer steps for every n, because $O(n)$ beats $O(n^2)$
- **c)** B does fewer steps for every n, because its constant is 1
- **d)** They are equally good, because both are polynomials

W2-M30 [why] Which sentence about linear search is **correct**?

- **a)** Its worst case is $\Theta(1)$, because Θ describes the best input
- **b)** Big-O means "worst case" and Big-Ω means "best case"
- **c)** Its best case is $\Theta(1)$ and its worst case is $\Theta(n)$
- **d)** It does exactly n comparisons on every input of size n

W2-M31 [how] What is the running time of this loop?

```
i = 0
while i * i < n:
    i += 1
```

- **a)** $\Theta(\log n)$
- **b)** $\Theta(\sqrt{n})$
- **c)** $\Theta(n)$
- **d)** $\Theta(n^2)$

W2-M32 [why] A recursive `total(values)` makes one call per element and creates no new data. Its **extra space** is:

- **a)** $O(1)$, because it creates no new data
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n)$, because every call that has not returned keeps a frame on the call stack

W2-M33 [how] A partly filled `Array` has **size 7**. How many elements move during `remove_at(arr, 7, 2)`?

- **a)** 4
- **b)** 5
- **c)** 2
- **d)** 7

W2-M34 [why] `remove_at` ends with `arr[size - 1] = None`. Why?

- **a)** So that `len(arr)` goes down by one
- **b)** Without it, the shifted values would end up in the wrong order
- **c)** That slot is no longer in use; clearing it removes a stale copy of the last value, which would otherwise keep that object alive
- **d)** The course `Array` raises an error if a slot is not `None`

W2-M35 [what] In C, after `int a[6];`, the statement `a[6] = 0;`:

- **a)** raises an index error at run time
- **b)** makes the array grow to 7 slots
- **c)** is the same as `a[0] = 0`, because indices wrap round
- **d)** writes past the end with no check, over whatever is stored next — a buffer overflow

W2-M36 [why] One Python array can hold an `int`, a `str` and a `list`, and still index in $O(1)$. Why?

- **a)** Python sorts the objects by size first
- **b)** Each slot holds a **reference**, and all references are the same size
- **c)** Python pads every object to the size of the largest one
- **d)** It cannot: indexing is $O(1)$ only when all items have the same type

W2-M37 [what] `a = Array(3)`, then `a[0:2]`. What happens?

- **a)** `TypeError`: the course `Array` does not support slicing
- **b)** a new `Array` of two `None`s is returned
- **c)** the list `[None, None]` is returned
- **d)** `IndexError`: index 2 is the last slot

W2-M38 [how] `arr` holds `[1, 2, 3, 4, 5, 6]`. What does it hold after `rotate_left(arr, 8)`?

- **a)** `[1, 2, 3, 4, 5, 6]` — 8 is more than a full turn, so nothing moves
- **b)** nothing: it raises `IndexError`, since 8 > 6
- **c)** `[3, 4, 5, 6, 1, 2]`
- **d)** `[5, 6, 1, 2, 3, 4]`

W2-M39 [why] `merge_sorted(a, b)` takes the front of `a` when `a[i] <= b[j]`. Why <= rather

than <?

- **a)** With <, the result would not be sorted
- **b)** With <, two equal values would raise an error
- **c)** With <, the loop would never end on equal values
- **d)** With <=, equal values keep their order — the one from a first — which merge sort relies on to be stable

W2-M40 [how] A 3 × 4 matrix holding the values 0 to 11 row by row is stored flat in row-major order. `transpose_flat(arr, 3, 4)` returns the 4 × 3 transpose, also row-major. At which index of the result is the value **1**?

- **a)** 1
- **b)** 3
- **c)** 4
- **d)** 9

W2-M41 [why] "Any sort that works only by comparing elements needs $\Omega(n \log n)$ comparisons in the worst case." This means:

- **a)** no comparison sort can have a worst case below a constant times $n \log n$; counting sort escapes only because it does not compare
- **b)** every sorting algorithm, counting sort included, is at least $n \log n$
- **c)** every comparison sort does exactly $n \log n$ comparisons
- **d)** comparison sorts are $O(n \log n)$ in the worst case

W2-M42 [how] How many pairs does `is_sorted(arr)` compare when `arr` holds `[5, 1, 2, 3, 4, 6, 7, 8]`?

- **a)** 7
- **b)** 8
- **c)** 1
- **d)** 0

W2-M43 [how] What happens when `increment()` is called?

```
count = 0

def increment():
    count = count + 1
```

- **a)** the global `count` becomes 1
- **b)** `NameError`: `count` is not defined
- **c)** nothing visible: a local `count` becomes 1
- **d)** `UnboundLocalError`

W2-M44 [why] What does `print(users)` show?

```
users = ['guest1', 'guest2', 'ali', 'guest3']
for u in users:
    if u.startswith('guest'):
```

```
        users.remove(u)
print(users)
```

- **a)** `['ali']`
- **b)** `['guest2', 'ali']`
- **c)** `['guest2', 'ali', 'guest3']`
- **d)** `RuntimeError`: the list changed size during iteration

Part B — Short answer and essay

W2-E1 [why] *(4 marks)* Big-O throws away constant factors and lower-order terms. Explain *why* each is thrown away, give one situation where doing so is misleading, and explain how measuring complements counting.

W2-E2 [what] *(4 marks)* Define O, Ω and Θ precisely. For $f(n) = 3n^2 + 2n$, give one true statement using each, and one *true but uninformative* O statement.

W2-E3 [why] *(3 marks)* Describe the best, worst and average case of linear search, with the number of comparisons for each. Which one does "linear search is $O(n)$" usually refer to, and why is that the default?

W2-E4 [why] *(4 marks)* Explain why indexing an array is $O(1)$ but inserting into the middle is $O(n)$. What does the fact that an array has a *fixed size* force you to do when it fills up, and what does that cost?

W2-E5 [why] *(4 marks)* State the course storage rule from Lecture 02. Give the reason for it, list what remains allowed, and show how two structures from later in the course follow it.

W2-E6 [what] *(3 marks)* Compare Python's `list`, `array.array`, `numpy.ndarray` and a `ctypes` array on: can it grow, what it holds, and whether it hides costs. Why is the course `Array` built on `ctypes`?

W2-E7 [why] *(4 marks)* `values` is a Python list of n numbers. Give the running time of each numbered part of this function, justify each with one of the five rules, and give the total. Why is part (3) **not** $O(n^2)$, although it hides a slice and an `in` inside a loop?

```
def report(values):
    total = 0
    for v in values:                # (1)
        total += v
    i = len(values)
    while i > 1:                    # (2)
        i //= 2
    for v in values:                # (3)
        if v in values[:10]:
            total -= v
    return total
```

W2-E8 [why] *(4 marks)* Define the **auxiliary space** of an algorithm. Give the extra space of `reverse_in_place`, `resized(arr, 2n)`, `merge_sorted(a, b)` and a recursive `total(values)`, with a reason for each. Why does recursion use space even when it creates no data, and what limit does Python put on it?

W2-E9 [how] *(3 marks)* Explain row-major order. A 5 × 7 matrix (5 rows, 7 columns) is stored flat in row-major order: at which flat index is element (3, 4), and which (row, column) is at flat index 30? Why is reaching (r, c) still $O(1)$?

W2-E10 [why] *(4 marks)* The course `Array` refuses negative indices, slicing and append. For each refusal, explain why it is deliberate and what you write instead. Then explain the difference between `len(a)` and the **size** of a partly filled array, and who keeps track of each.

Part C — Complexity analysis

Give the running time in Θ notation as a function of n, and **justify** it in one or two sentences (which rule, which sum). Where best and worst differ, give both.

W2-K1 [how]

```
total = 0
for i in range(n):
    for j in range(n):
        total += i * j
for k in range(n):
    total += k
```

W2-K2 [how]

```
i = 1
while i < n:
    i = i * 2
```

W2-K3 [how]

```
for i in range(n):
    j = n
    while j > 1:
        j = j // 2
```

W2-K4 [how] (`arr` is an `Array` of n elements.)

```
def has_duplicate(arr):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

W2-K5 [how] (`values` is a Python list of n items.)

```
for i in range(n):
    rest = values[i:]
```

W2-K6 [how]

```
count = 0
for i in range(n):
    for j in range(100):
        for k in range(i):
            count += 1
```

Part D — Proofs

W2-P1 [how] *(3 marks)* Prove that $2n + 10 = O(n)$ by giving c and n_0 and showing the inequality.

W2-P2 [why] *(3 marks)* Prove that n^2 is **not** $O(n)$.

W2-P3 [how] *(4 marks)* Prove that $4n^2 + n = \Theta(n^2)$.

Part E — Array state

Draw the array after each step (use _ for an unused slot) and count the elements moved.

W2-S1 [how] An `Array` of capacity 7 holds `[10, 20, 30, 40, 50, _, _]`, size 5.

(a) `size = insert_at(arr, size, 1, 15)`
(b) then `remove_at(arr, size, 3)` (and `size` decreases by one)

W2-S2 [why] A student writes the shifting loop in insertion **forwards**:

```
for i in range(index, size):
    arr[i + 1] = arr[i]
arr[index] = value
```

Show the array after inserting `"X"` at index 1 into `["a", "b", "c", "d", _]`, size 4. What went wrong, and what is the fix?

W2-S3 [how] `rotate_left` by three reversals. Show the array after each reversal when rotating `[1, 2, 3, 4, 5, 6]` left by 2.

W2-S4 [how] The 3 × 4 matrix holding the values 0 to 11 row by row is stored flat in row-major order.

(a) Which value is at (1, 3)? At which flat index?
(b) Which (row, column) is flat index 10?

(c) After `transpose_flat` (now 4 × 3, row-major), at which flat index is the value that was at (1, 3)?

Part F — Trace the code

Write exactly what is printed.

W2-T1 [how]

```
for n in range(10, 16):
    for d in range(2, n):
        if n % d == 0:
            break
    else:
        print(n, end=" ")
print("done")
```

W2-T2 [how]

```
def run(command):
    match command.split():
        case ["add", a, b]:
            return int(a) + int(b)
        case ["neg", a]:
            return -int(a)
        case ["sum", *rest]:
            return len(rest)
        case _:
            return "?"

print(run("add 2 3"), run("neg 4"), run("sum"), run("sum 1 2 3"), run("add 1"))
```

W2-T3 [how]

```
def f(x):
    try:
        print("A", end=" ")
        y = 10 // x
        print("B", end=" ")
    except ZeroDivisionError:
        print("C", end=" ")
        return 0
    else:
        print("D", end=" ")
        return y
    finally:
```

```
        print("E", end=" ")

print(f(2))
print(f(0))
```

W2-T4 [how]

```
def change(values, n):
    values.append(n)
    n = n * 10
    values = [0]
    return n

data, k = [1], 2
result = change(data, k)
print(data, k, result)
```

Part G — Find and fix the bug

W2-B1 [how] Should return `True` exactly when n is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5)):
        if n % d == 0:
            return False
    return True
```

W2-B2 [how] Should return the index of `target` in `arr`, or −1.

```
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
        else:
            return -1
```

W2-B3 [how] Should remove and return `arr[index]` from a partly filled array, shifting the rest left.

```
def remove_at(arr, size, index):
    value = arr[index]
    for i in range(index, size):
        arr[i] = arr[i + 1]
    return value
```

Part H — Write the code

In `practice/week02.py`; check with `pytest tests/test_practice_week02.py -v`. Work on the `Array` with indices only, and meet the stated complexity.

W2-C1 [how] `count_occurrences(arr, target, size=None)` — O(n).

W2-C2 [why] `remove_all(arr, size, target)` — remove every occurrence in **one pass**, O(n) time and O(1) extra space; return the new size. Why is calling `remove_at` once per occurrence $O(n^2)$?

W2-C3 [why] `two_sum_sorted(arr, target)` — in a **sorted** array, find i < j with `arr[i] + arr[j] == target` in O(n). Why is it correct to move only one index at each step?

W2-C4 [how] `prefix_sums(arr)` — a new `Array` with running totals, O(n). How does it let you answer "sum of `arr[i..j]`" in O(1)?